

UNITREE A1

ROBOT DOG

With **NVIDIA Omniverse** and

Isaac Sim



PRESENTED BY

Swan Pyae Sone

65011683

Thura Zaw

65011685

ABOUT REINFORCEMENT LEARNING

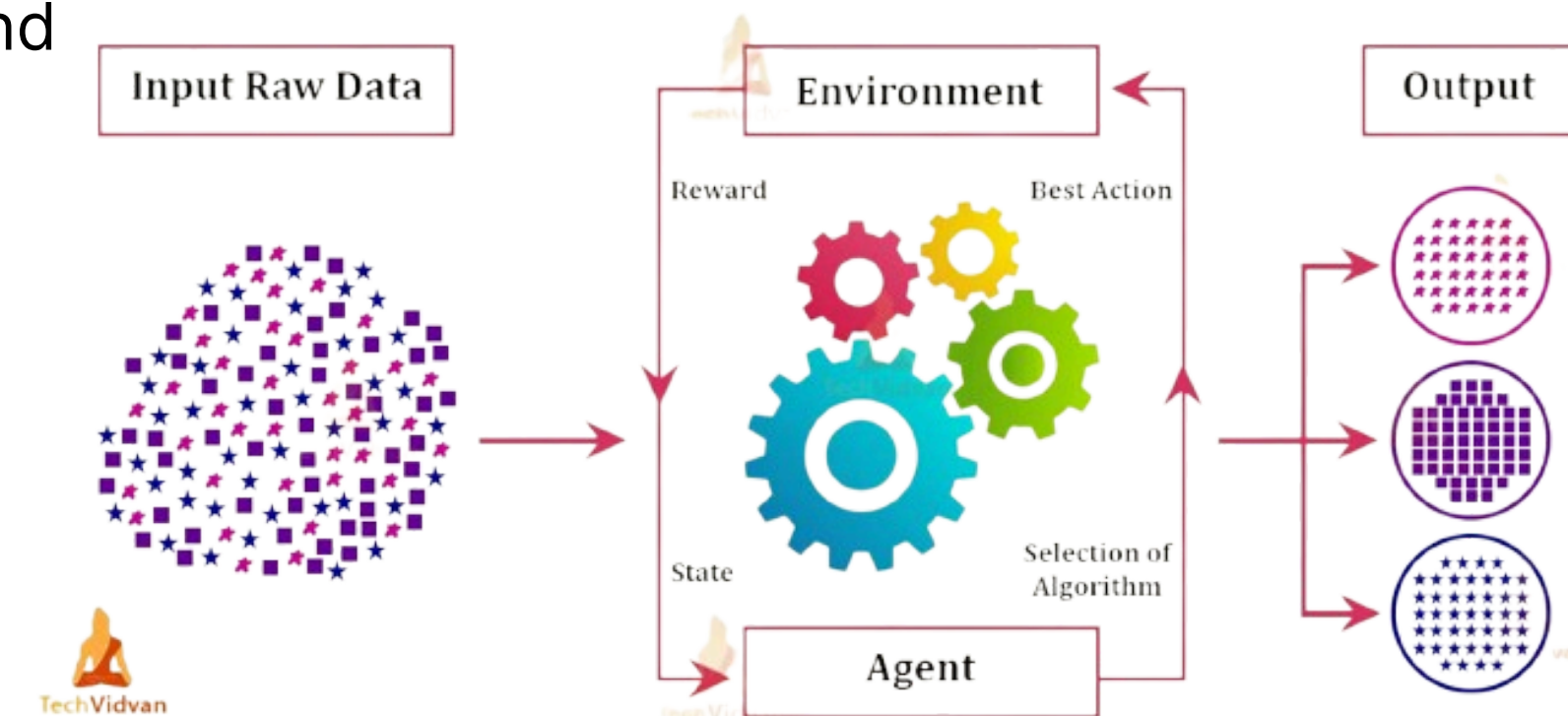
What is Reinforcement Learning (RL)?

- A type of machine learning where an agent learns by trial and error in an environment to maximize a reward.

Core Elements:

- Agent: The decision-maker (e.g., robot dog).
- Environment: The setting the agent interacts with (e.g., virtual world in Isaac Sim).
- Action: Choices the agent makes.
- Reward: Feedback based on action quality (positive for good, negative for bad).

Reinforcement Learning in ML



REQUIRE **TOOLS**

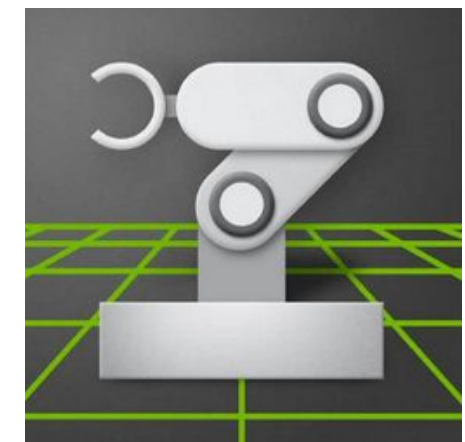
Omniverse

Omniverse is an open platform developed by NVIDIA for real-time simulation and collaboration across 3D workflows. It acts as a collaborative hub that integrates diverse tools, datasets, and applications in a single environment.



Isaac Sim

Isaac Sim is part of the Omniverse platform, specifically designed for robotics simulation. It enables training and testing of robots in a high-fidelity, physics-rich virtual environment.

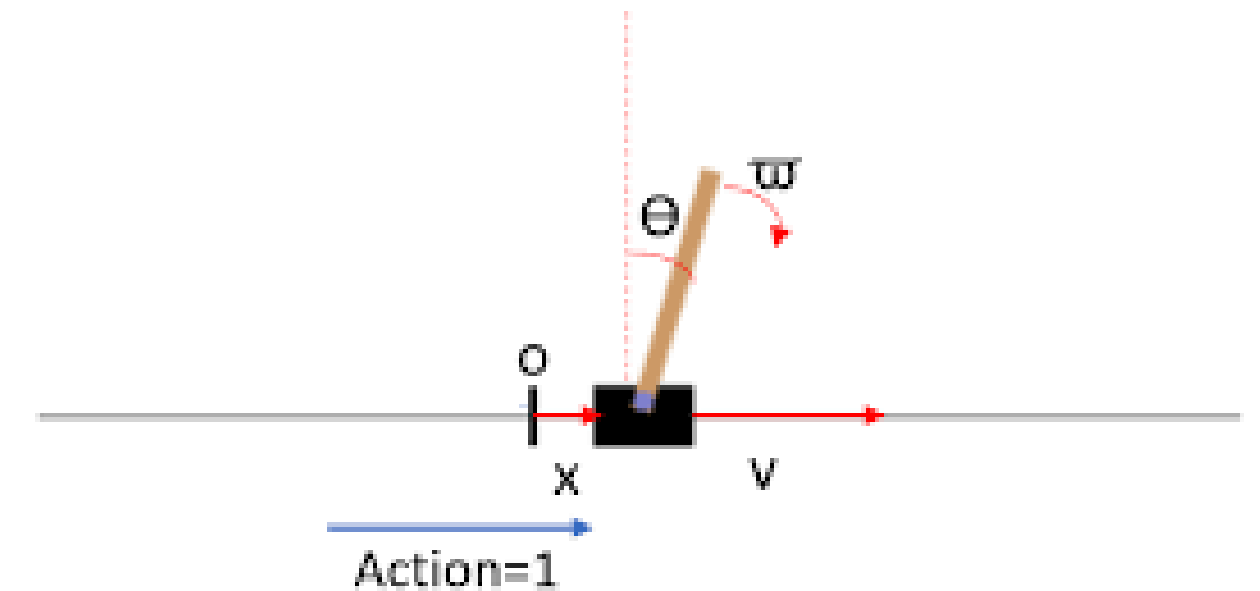


STATE SPACE

Discrete

- **CartPole-v1**: The agent controls a cart that can move left or right to keep a pole balanced on top. The action space is limited to two options: move left or move right.
- **FrozenLake-v1**: A grid-based environment where the agent must navigate across a frozen lake. Actions are discrete (e.g., move up, down, left, or right).

frame: 53, Obs: (0.018, 0.669, 0.286, 0.618)
Action: 1.0, Cumulative Reward: 47.0, Done: 1



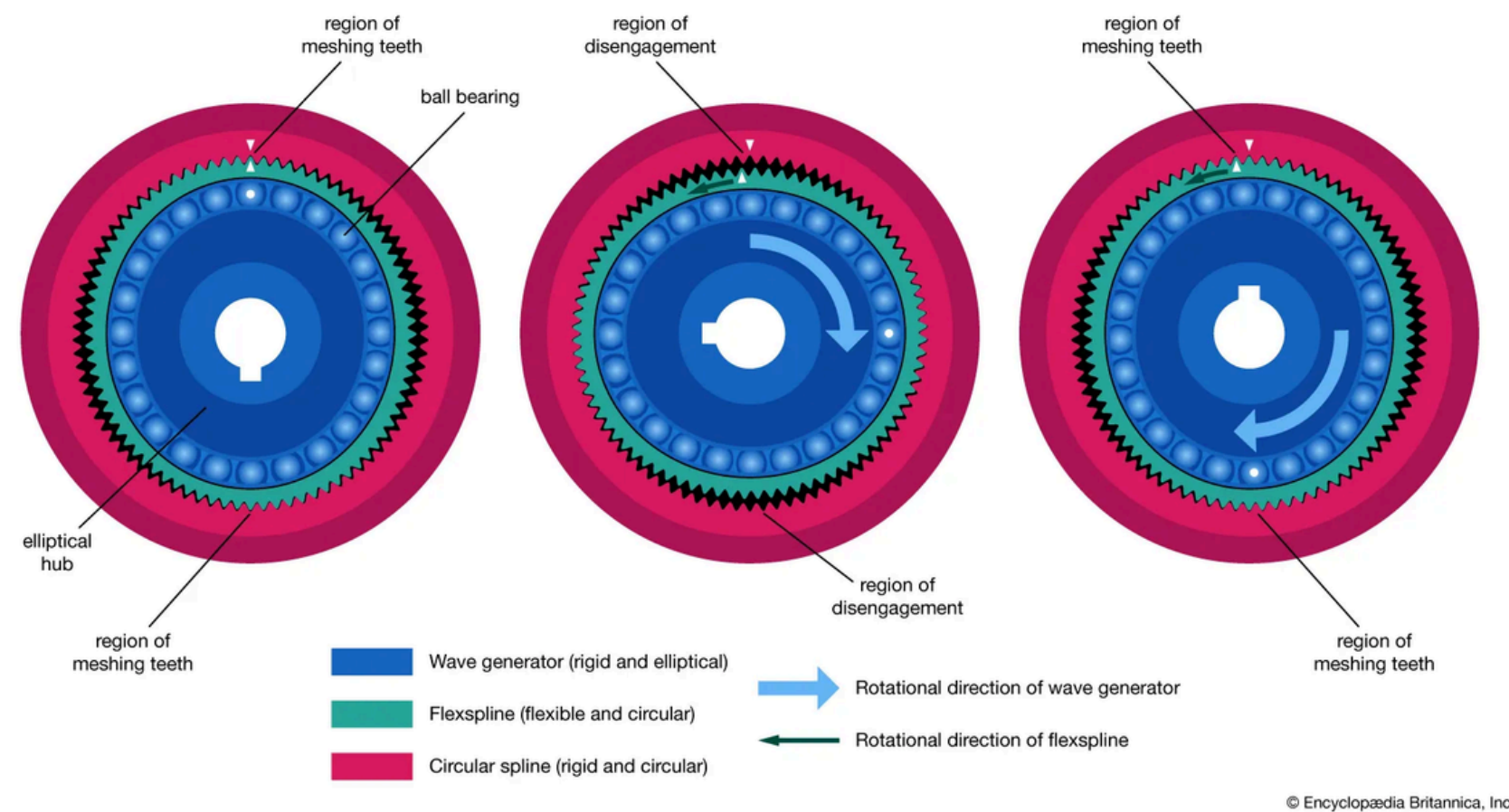
Continuous

- **Pendulum-v1**: The agent controls the torque applied to a pendulum to swing it upright. The action space is continuous, representing a range of torques.
- **Hopper-v3**: The agent controls a one-legged hopping robot, adjusting joint angles in a continuous space to achieve balance and forward movement.



IN OUR CASE

Harmonic Drive



- **Smooth Movement:** Walking, running, and balancing require the robot to make small, gradual adjustments to each joint's angle and speed. Continuous control allows the robot to vary these parameters seamlessly, enabling natural, fluid movements that would be difficult to achieve with discrete actions.
- **Precision in Joint Angles and Torque:** Each motor can adjust to a precise position and apply a specific torque value, which is necessary for accurate control over each joint. In discrete control, options would be limited to a few fixed positions or forces, reducing flexibility and accuracy.

STATE SPACE

- **Base Linear Velocity:** Tracks the robot's overall movement speed and direction in 3D, allowing it to assess how quickly it's advancing or retreating.
- **Base Angular Velocity:** Measures the rotation rate of the robot's base, providing feedback on shifts in orientation and helping maintain directional control.
- **Projected Gravity Vector:** Indicates the robot's orientation relative to gravity, essential for balance and detecting potential tipping.
- **Joint Angles:** Captures each joint's position, giving the robot an understanding of its current pose.
- **Joint Speed:** Monitors the rate of motion at each joint, helping the robot fine-tune its movements for smoother adjustments.
- **Last Action Executed:** Remembers the previous movement command, helping the robot achieve continuity in movement and refine stability.

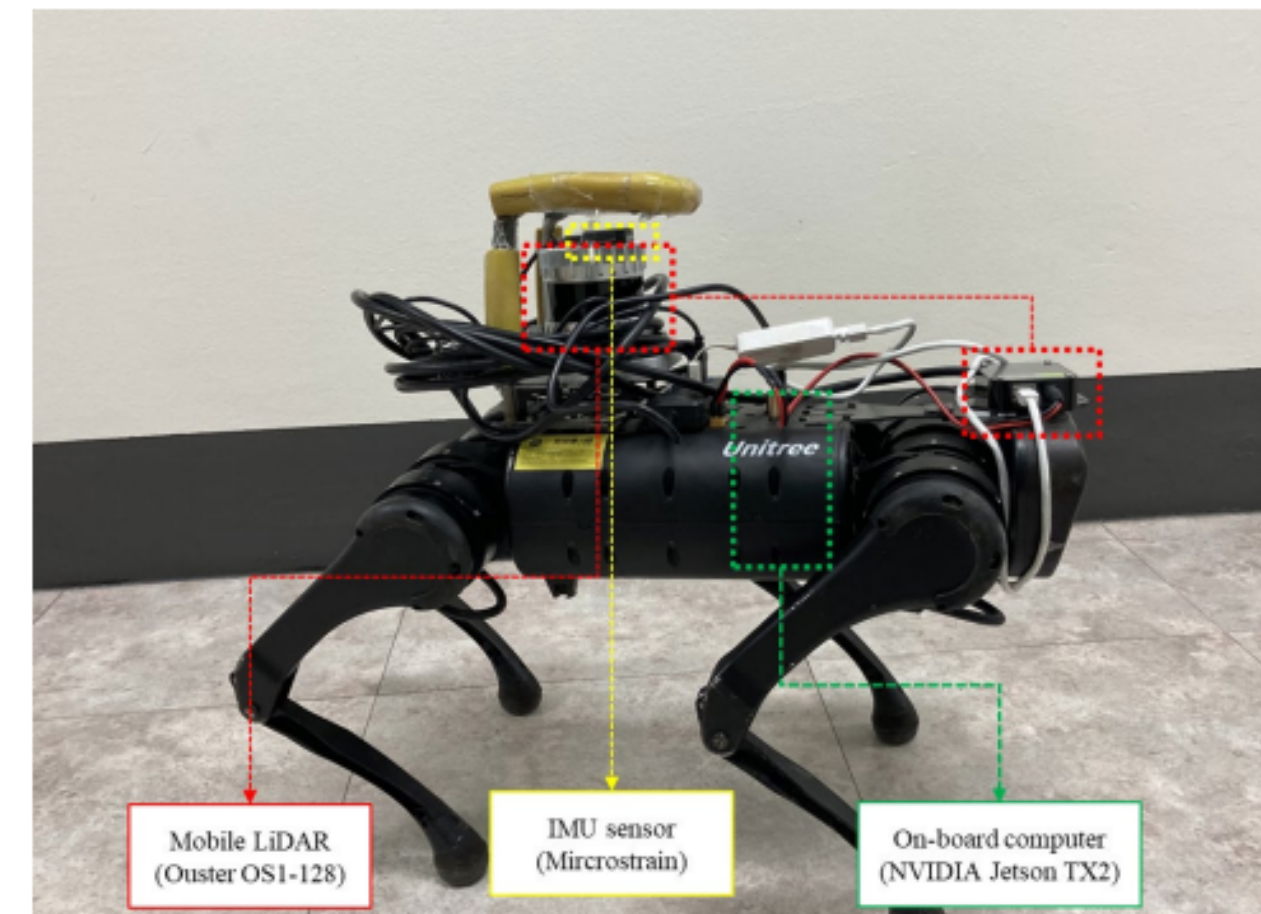
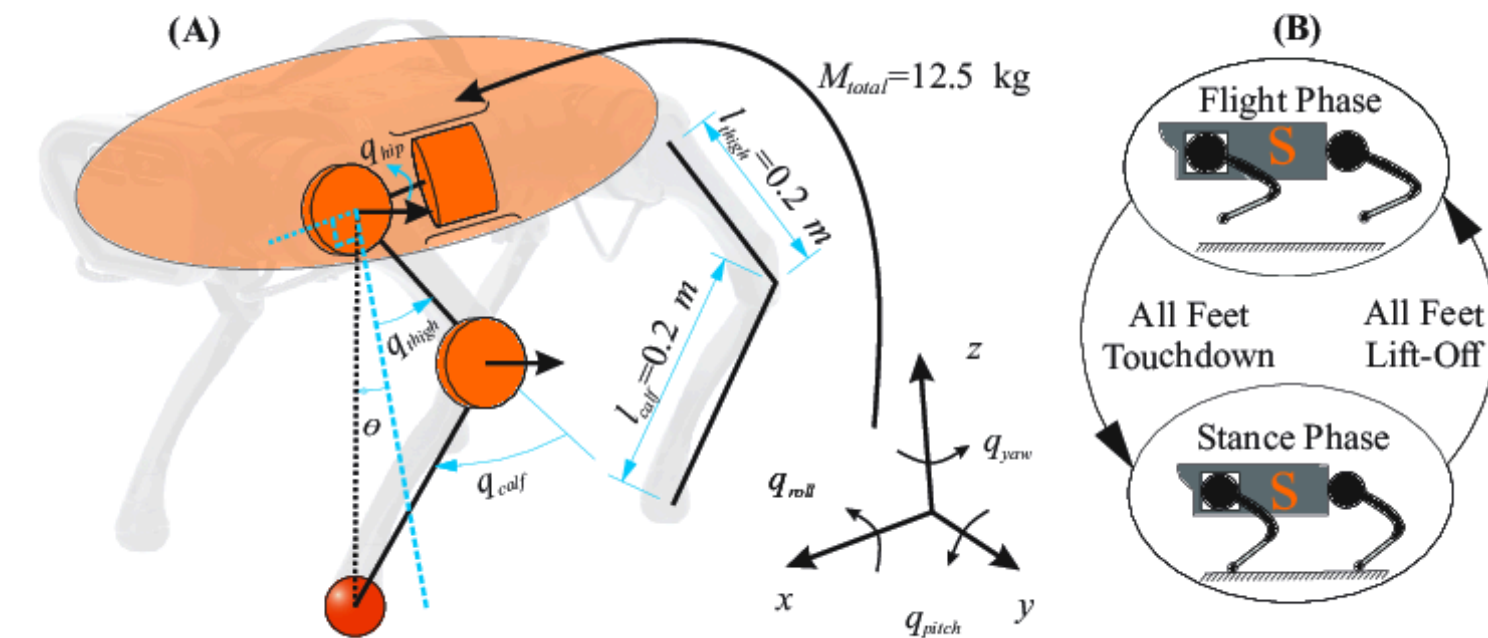


ACTION SPACE

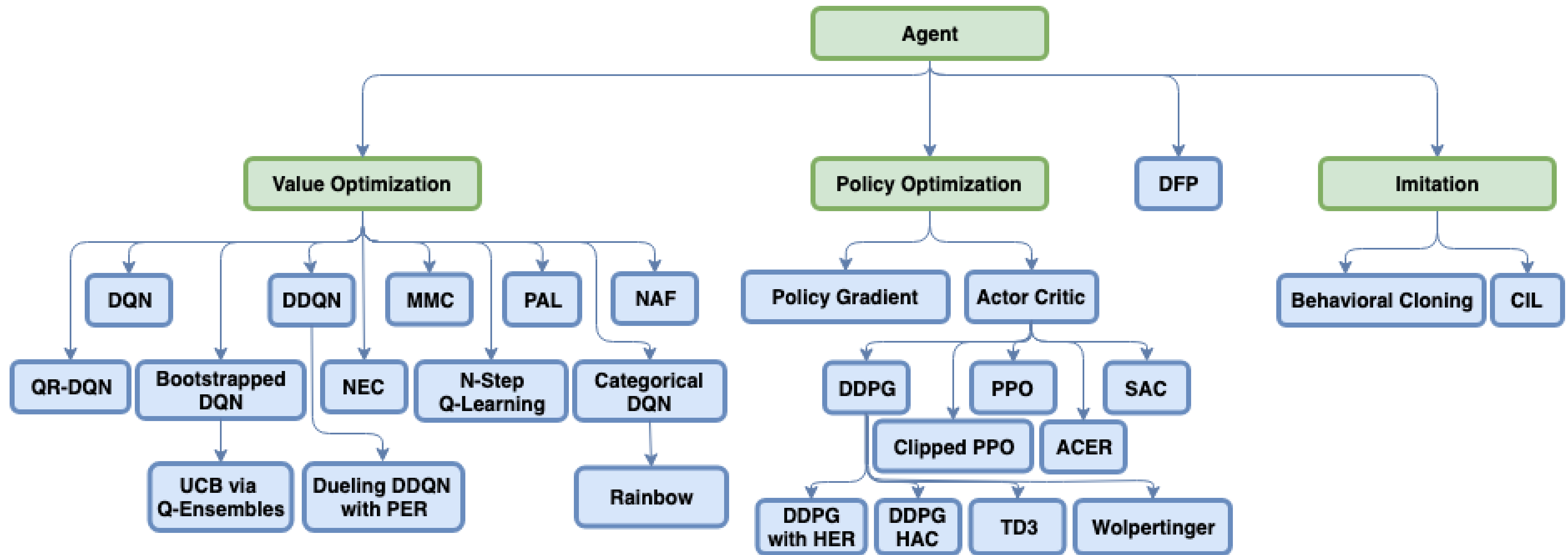
- **Hip Joint:** Enables side-to-side movement, allowing the robot to shift its weight and stabilize; primarily rotates around the x-axis.
- **Thigh Joint:** Drives forward and backward motion, managing the robot's stride length and direction; rotates around the y-axis to facilitate walking or running.
- **Calf Joint:** Controls leg bending and straightening, critical for lifting or lowering the leg; rotates around the y-axis to handle leg extension and flexion.

Additional Joint

- **IMU Joint:** The IMU provides data on the robot's orientation, acceleration, and angular velocity.
- **Camera Joint:** The camera acts as the robot's visual sensor, capturing images or video to recognize objects, obstacles, and the environment.



FUNDAMENTAL RL ALGORITHMS



OUR TASKS



Crouch to Stand



Walking

DDPG FOR CONTINUOUS CONTROL

Why Standard RL Isn't Enough for Robotics

- **Precise, Continuous Actions Needed:** Robots, like a robot dog, require fine-grained control over joints and movements, which typical RL methods don't handle well.
- **Continuous Action Space:** Unlike discrete actions (e.g., moving left or right), robotics involves smooth, continuous actions (e.g., moving an arm to an exact position).

Solution: Deep Deterministic Policy Gradient (DDPG)

- **Designed for Continuous Control:** DDPG is an RL algorithm specifically built for environments where actions are continuous, like robotic movement.
- **Actor-Critic Structure:** Uses two networks:
 - Actor: Chooses actions.
 - Critic: Evaluates action quality (Q-value).
- DDPG enables the robot dog to learn natural, smooth movements, crucial for tasks like walking and balancing.



OVERVIEW OF DDPG (DEEP DETERMINISTIC POLICY GRADIENT)

What is DDPG?

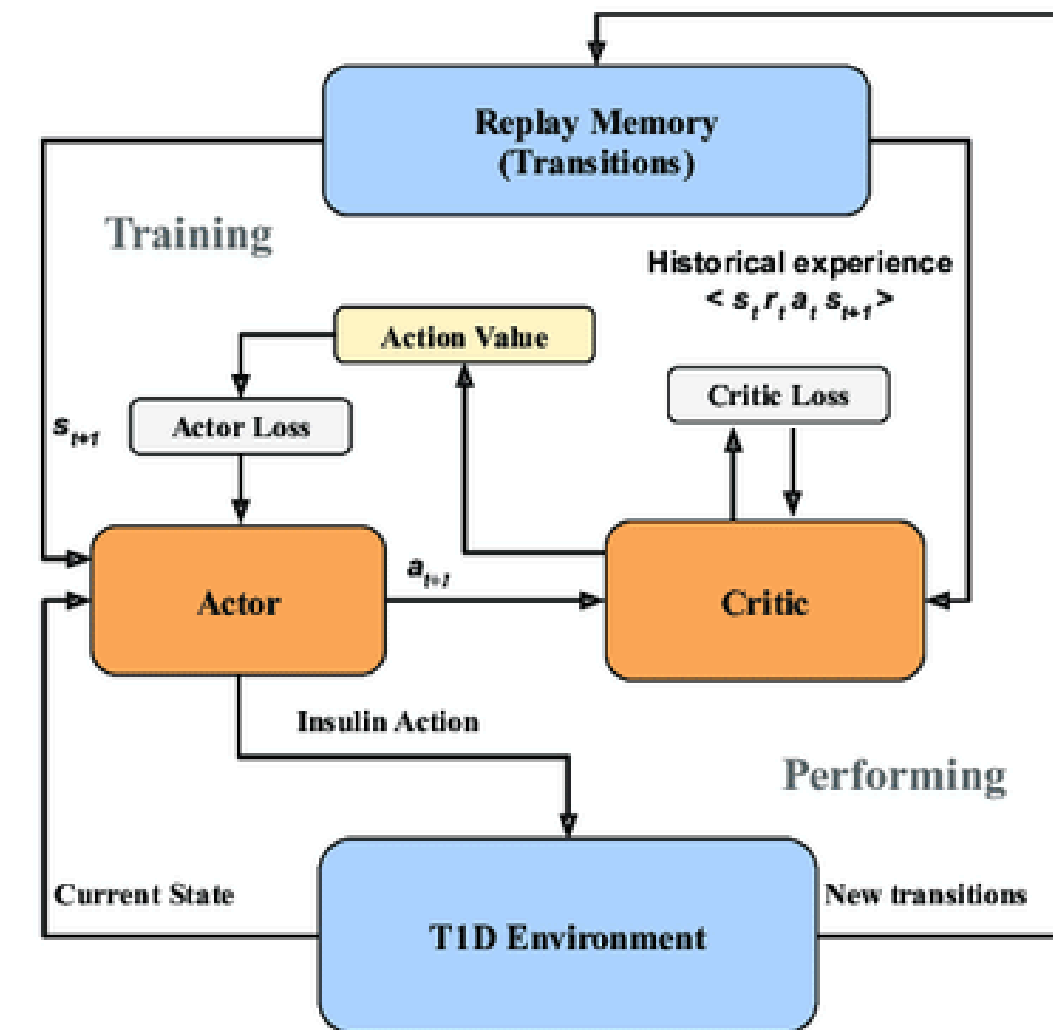
- A popular reinforcement learning algorithm for continuous action spaces.
- Combines ideas from Deep Q-Networks (DQN) and actor-critic methods to allow precise, stable control in complex environments.

Core Components

- **Actor-Critic Network:** The actor decides on actions, while the critic evaluates them based on rewards.
- **Replay Buffer:** Stores past experiences to sample from, breaking sequence dependency and stabilizing learning.
- **Explicit noise:** In DDPG promotes exploration by adding randomness to actions, allowing the agent to try varied behaviors in continuous spaces.
- **Target Networks:** Slow-moving versions of the actor and critic networks to reduce drastic learning changes.

Why DDPG is Ideal for the A1 Robo Dog?

- **Continuous Action Control:** Fine-grained control over actions, which is crucial for smooth and adaptive movements.
- **Stability and Efficiency:** The replay buffer and target networks make it stable and efficient, suitable for robotic tasks that require reliability.



CORE COMPONENTS

1. Actor-Critic Network

Actor Network

- **Role:** Chooses actions based on the current state.
- **Function:** The actor outputs a specific action that the agent should take, aiming to maximize future rewards.
- **Purpose:** It learns a policy by mapping each state to the best action, guiding the agent's behavior.

Critic Network

- **Role:** Evaluates the action chosen by the actor.
- **Function:** The critic predicts a Q-value (expected reward) for the state-action pair, estimating how good the chosen action is in the current situation.
- **Purpose:** It helps the actor improve by providing feedback on the quality of actions, which the actor uses to adjust and find better actions over time.

```
class Actor(nn.Module):
    def __init__(self, state_size, action_size):
        super(Actor, self).__init__()
        self.fc1 = nn.Linear(state_size, 256)
        self.fc2 = nn.Linear(256, 256)
        self.fc3 = nn.Linear(256, action_size)

    def forward(self, state):
        x = torch.relu(self.fc1(state))
        x = torch.relu(self.fc2(x))
        # Output joint torques
        actions = torch.tanh(self.fc3(x))
        return actions

class Critic(nn.Module):
    def __init__(self, state_size, action_size):
        super(Critic, self).__init__()
        self.fc1 = nn.Linear(state_size + action_size, 256)
        self.fc2 = nn.Linear(256, 256)
        self.fc3 = nn.Linear(256, 1)

    def forward(self, state, action):
        x = torch.cat([state, action], dim=1)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        value = self.fc3(x)
        return value
```

CORE COMPONENTS

2. Replay Buffer

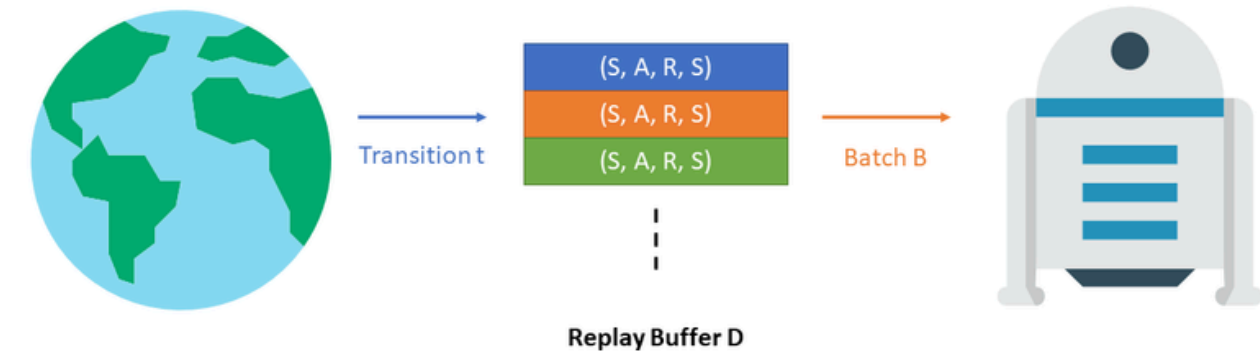
The Replay Buffer in DDPG is a memory storage that holds past experiences, which are recorded as (state, action, reward, next state) tuples.

Why Replay Buffer is Used

- **Breaks Correlation:** Random sampling reduces correlations between consecutive actions, stabilizing learning.
- **Sample Efficiency:** Allows the agent to reuse past experiences, enhancing training efficiency.
- **Balanced Learning:** Facilitates learning from both successes and mistakes across varied situations.

Experience Replay

- Save transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ into buffer and sample batch B
- Use batch B to train the agent



CORE COMPONENTS

3. Explicit noise

- **Purpose:** Encourages exploration by adding randomness to the actions selected by the actor.
- **Function:** Helps the agent try varied actions rather than always choosing the same deterministic action, which is crucial for discovering better strategies.
- **Implementation:** Often achieved using the Ornstein-Uhlenbeck process for smooth, temporally correlated noise, suitable for continuous action spaces.

Types of Noise Used in DDPG

- **Ornstein-Uhlenbeck Noise**
 - The noise is smooth and resembles how physical systems might behave, allowing the agent to make gradual adjustments to actions, which is essential for achieving realistic movements.
- **Gaussian Noise:**
 - Another option is Gaussian noise, where random values are sampled from a normal distribution.

CORE COMPONENTS

4. Target Networks

- **Purpose:** Target networks stabilize the training process by providing a steady reference for updating the main networks, helping avoid instability and sudden changes.
- **Target networks** reduce oscillations and errors in value estimation, leading to smoother training.
- They allow the agent to incrementally adjust its policy without abrupt shifts, helping the model converge on optimal actions more reliably.

```
# Update target networks
for param, target_param in zip(self.actor.parameters(), self.target_actor.parameters()):
    target_param.data.copy_(TAU * param.data + (1 - TAU) * target_param.data)

for param, target_param in zip(self.critic.parameters(), self.target_critic.parameters()):
    target_param.data.copy_(TAU * param.data + (1 - TAU) * target_param.data)
```

CONCEPT TO REAL

1.Import Physics and Isaac Sim Asserts

```
from omni.isaac.core import World
from omni.isaac.core.utils.prims import define_prim, get_prim_at_path
from omni.isaac.nucleus import get_assets_root_path
from omni.isaac.quadruped.robots import Unitree
from omni.isaac.core.articulations import ArticulationView
from omni.isaac.core.utils.types import ArticulationAction
```

2.State-Action Space Definition

```
# Define observation and action spaces
self.state_size = 27 # Joint positions (12) + velocities (12) + robot height + robot orientation(x , y)
self.action_size = 12 # Joint torques for all 12 joints

self.target_height = np.random.uniform(0.4, 0.6) # Target standing height in meters
```

3.Crouch Position

```
# Define crouching joint positions
self.default_dof_pos = {}

# Front Left Leg
"FL_hip_joint": 0.0, # Neutral hip position
"FL_thigh_joint": 0.8, # Thigh bent forward
"FL_calf_joint": 0.2, # Calf bent back

# Front Right Leg
"FR_hip_joint": 0.0, # Neutral hip position
"FR_thigh_joint": 0.8, # Thigh bent forward
"FR_calf_joint": -0.2, # Calf bent back

# Rear Left Leg
"RL_hip_joint": 0, # Neutral hip position
"RL_thigh_joint": 0, # Thigh bent forward
"RL_calf_joint": 0, # Calf bent back

# Rear Right Leg
"RR_hip_joint": 0, # Neutral hip position
"RR_thigh_joint": 0, # Thigh bent forward
"RR_calf_joint": 0 # Calf bent back
```

4.Get Observation

```
def _get_observation(self):
    # Get robot state
    joint_positions = self.al.get_joint_positions()
    joint_velocities = self.al.get_joint_velocities()
    robot_position = self.al.get_world_pose()[0]
    robot_orientation = self.al.get_world_pose()[1]

    # Calculate roll (x) and pitch (y) from quaternion
    sinr_cosp = 2 * (robot_orientation[0] * robot_orientation[1] + robot_orientation[2] * robot_orientation[3])
    cosr_cosp = 1 - 2 * (robot_orientation[1] * robot_orientation[1] + robot_orientation[2] * robot_orientation[2])
    roll = np.arctan2(sinr_cosp, cosr_cosp) # x-axis orientation

    sinp = 2 * (robot_orientation[0] * robot_orientation[2] - robot_orientation[3] * robot_orientation[1])
    pitch = np.arcsin(np.clip(sinp, -1.0, 1.0)) # y-axis orientation

    # Combine into state vector
    state = np.concatenate([
        joint_positions, # 12 values
        joint_velocities, # 12 values
        [robot_position[2]], # Height (1 value)
        [roll], # X-axis orientation (1 value)
        [pitch], # Y-axis orientation (1 value)
    ])
    return state
```

CONCEPT TO REAL

5. Reward System

```
def _calculate_reward(self, state):
    # Extract state information
    joint_positions = state[:12] # First 12 elements are joint positions
    joint_velocities = state[12:24] # Next 12 elements are joint velocities
    height = state[-2] # Robot height
    orientation = state[-1] # Robot orientation (yaw/z-axis)

    # Get roll and pitch from world pose
    quaternion = self.al.get_world_pose()[1] # [w, x, y, z]

    # Calculate roll (x-axis) and pitch (y-axis) from quaternion
    sinr_cosp = 2 * (quaternion[0] * quaternion[1] + quaternion[2] * quaternion[3])
    cosr_cosp = 1 - 2 * (quaternion[1] * quaternion[1] + quaternion[2] * quaternion[2])
    roll = np.arctan2(sinr_cosp, cosr_cosp)

    sinp = 2 * (quaternion[0] * quaternion[2] - quaternion[3] * quaternion[1])
    if abs(sinp) >= 1:
        pitch = np.copysign(np.pi / 2, sinp)
    else:
        pitch = np.arcsin(sinp)

    # 1. Height reward - encourage reaching target height
    height_error = abs(height - self.target_height)
    height_reward = -2.0 * height_error

    # 2. Enhanced orientation reward - now includes roll and pitch penalties
    orientation_reward = (-1.0 * abs(orientation) + # original yaw penalty
        -2.0 * abs(roll) + # roll penalty
        -2.0 * abs(pitch)) # pitch penalty
```

```
def _calculate_reward(self, state):
    # 3. Joint symmetry reward - fixed indexing for left-right comparison
    left_right_diff = (
        abs(joint_positions[0] - joint_positions[3]) + # Front legs hip
        abs(joint_positions[1] - joint_positions[4]) + # Front legs thigh
        abs(joint_positions[2] - joint_positions[5]) + # Front legs calf
        abs(joint_positions[6] - joint_positions[9]) + # Rear legs hip
        abs(joint_positions[7] - joint_positions[10]) + # Rear legs thigh
        abs(joint_positions[8] - joint_positions[11]) # Rear legs calf
    )
    symmetry_reward = -0.5 * left_right_diff

    # 4. Energy efficiency reward
    velocity_penalty = -0.01 * np.sum(np.square(joint_velocities))
    joint_limit_penalty = -0.01 * np.sum(np.square(joint_positions))

    # 5. Stability reward - comparing diagonal legs
    diagonal_balance = (
        abs((joint_positions[0] + joint_positions[11]) -
            (joint_positions[3] + joint_positions[8])) +
        abs((joint_positions[1] + joint_positions[10]) -
            (joint_positions[4] + joint_positions[7])) +
        abs((joint_positions[2] + joint_positions[9]) -
            (joint_positions[5] + joint_positions[6]))
    )
    stability_reward = -0.5 * diagonal_balance
```

```
# Combine all rewards with weights
reward = (height_reward * 0.8 +
    orientation_reward * 0.8 +
    symmetry_reward * 0.6 +
    velocity_penalty * 0.4 +
    joint_limit_penalty * 0.3 +
    stability_reward * 0.7)

# Optional early success bonus - now checks roll and pitch too
if (height_error < 0.02 and # Close to target height
    abs(orientation) < 0.1 and # Nearly level in yaw
    abs(roll) < 0.1 and # Nearly level in roll
    abs(pitch) < 0.1 and # Nearly level in pitch
    left_right_diff < 0.1): # Symmetric pose
    reward += 10.0 # Bonus for achieving stable standing

return reward
```

CONCEPT TO REAL

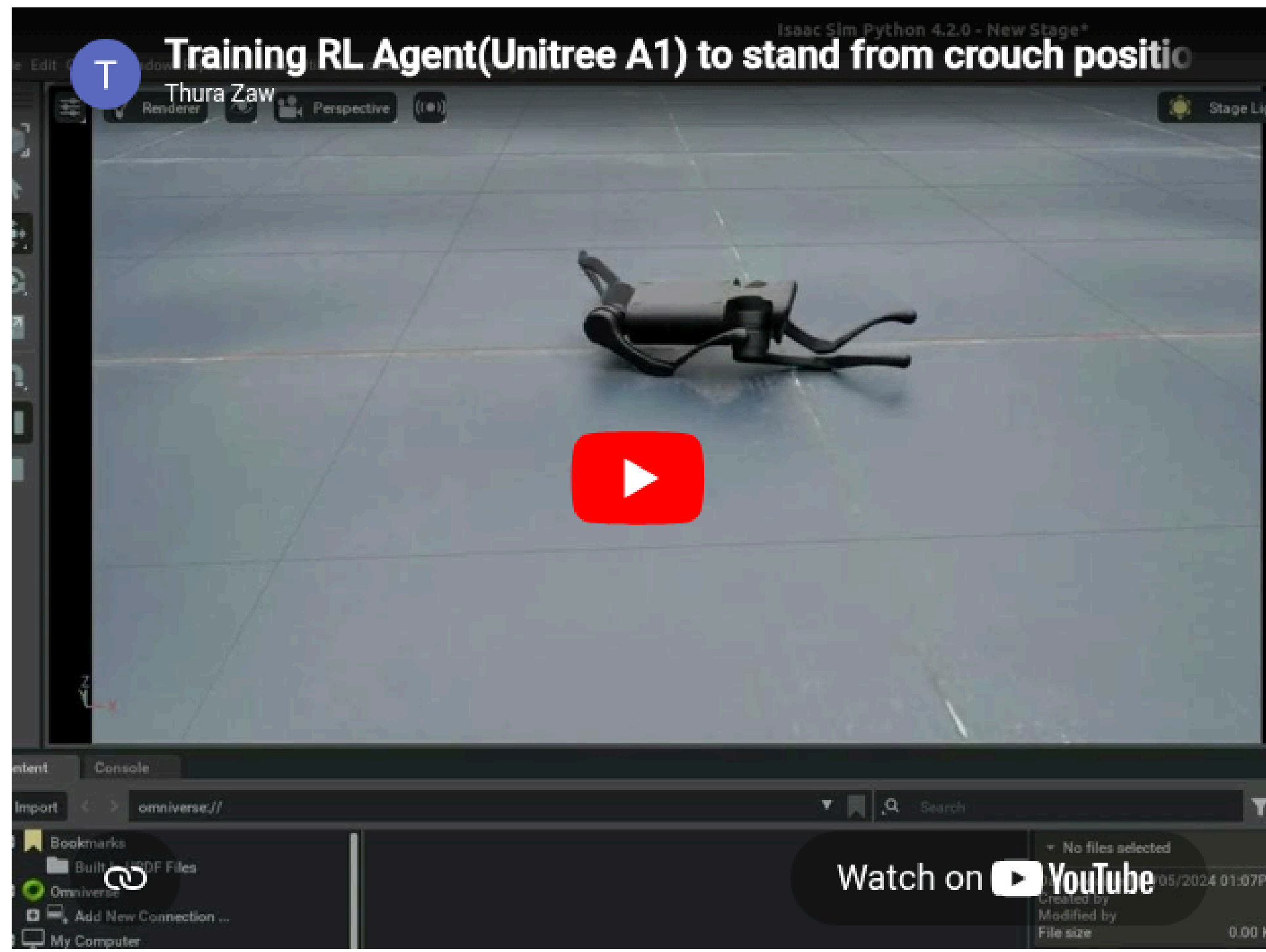
6. Terminate State

```
def _is_done(self, state):
    height = state[24] # Updated index for height
    roll = state[25]   # X-axis orientation
    pitch = state[26]  # Y-axis orientation

    # End episode if robot falls or achieves goal
    if height < 0.1: # Robot has fallen
        print("robot fallen")
        return True
    if (abs(height - self.target_height) < 0.02 and
        abs(roll) < 0.1 and
        abs(pitch) < 0.1):
        # Robot has achieved target pose
        print("target pose reached")
        return True
    if abs(roll) > 0.7 or abs(pitch) > 0.7 : # Robot is too tilted in any direction
        print("robot too tilted")
        return True

    return False
```

TRAINING PROGRESS



PROXIMAL POLICY OPTIMIZATION (PPO)

Proximal Policy Optimization or **PPO**, is policy-based reinforcement learning algorithm. It aims to improve learning stability and efficiency while balancing exploration and exploitation. PPO is reliable due to its robustness, simplicity, and effectiveness across a variety of tasks.

How PPO Fits into Training the A1 Robo Dog

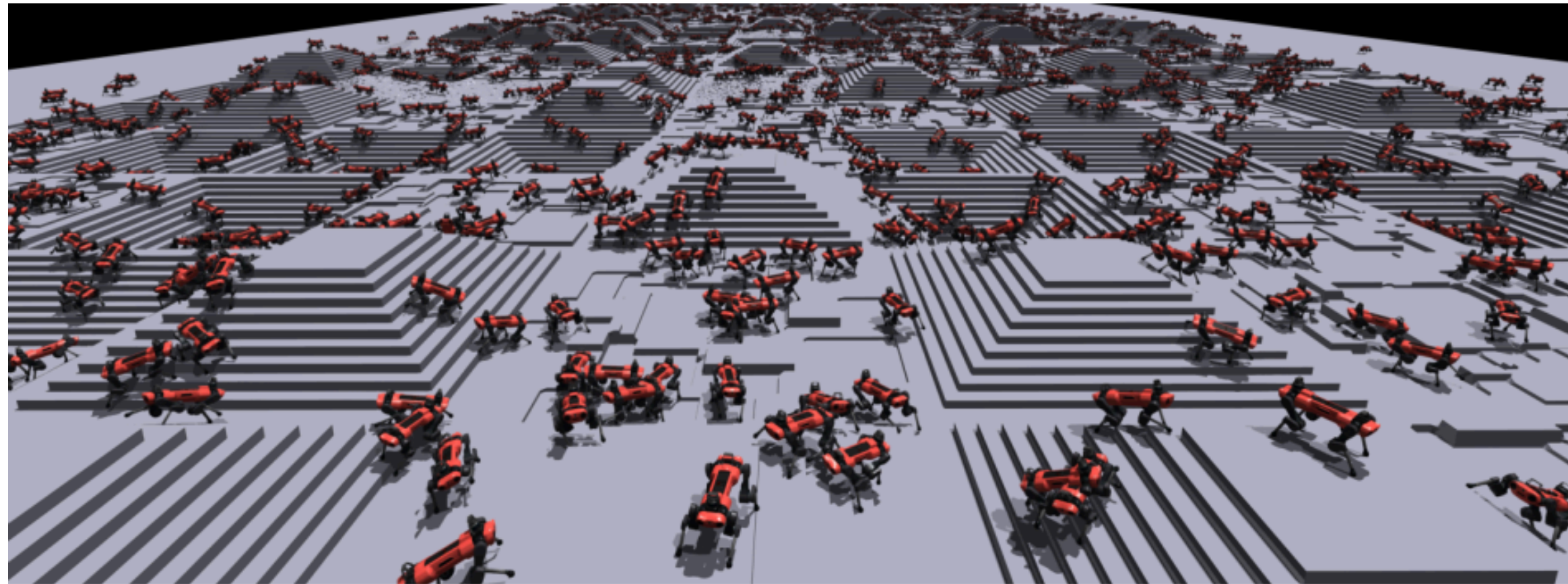
- Stable Learning in Simulation
- Policy Optimization with Clipping
- Adaptive KL Divergence
- Generalized Advantage Estimation (GAE)

Advantages of Using PPO for the A1 Robo Dog in Omniverse with Isaac Sim

- Robustness
- Adaptability
- Compatibility with Continuous Action Spaces

ENHANCED TRAINING METHODOLOGY

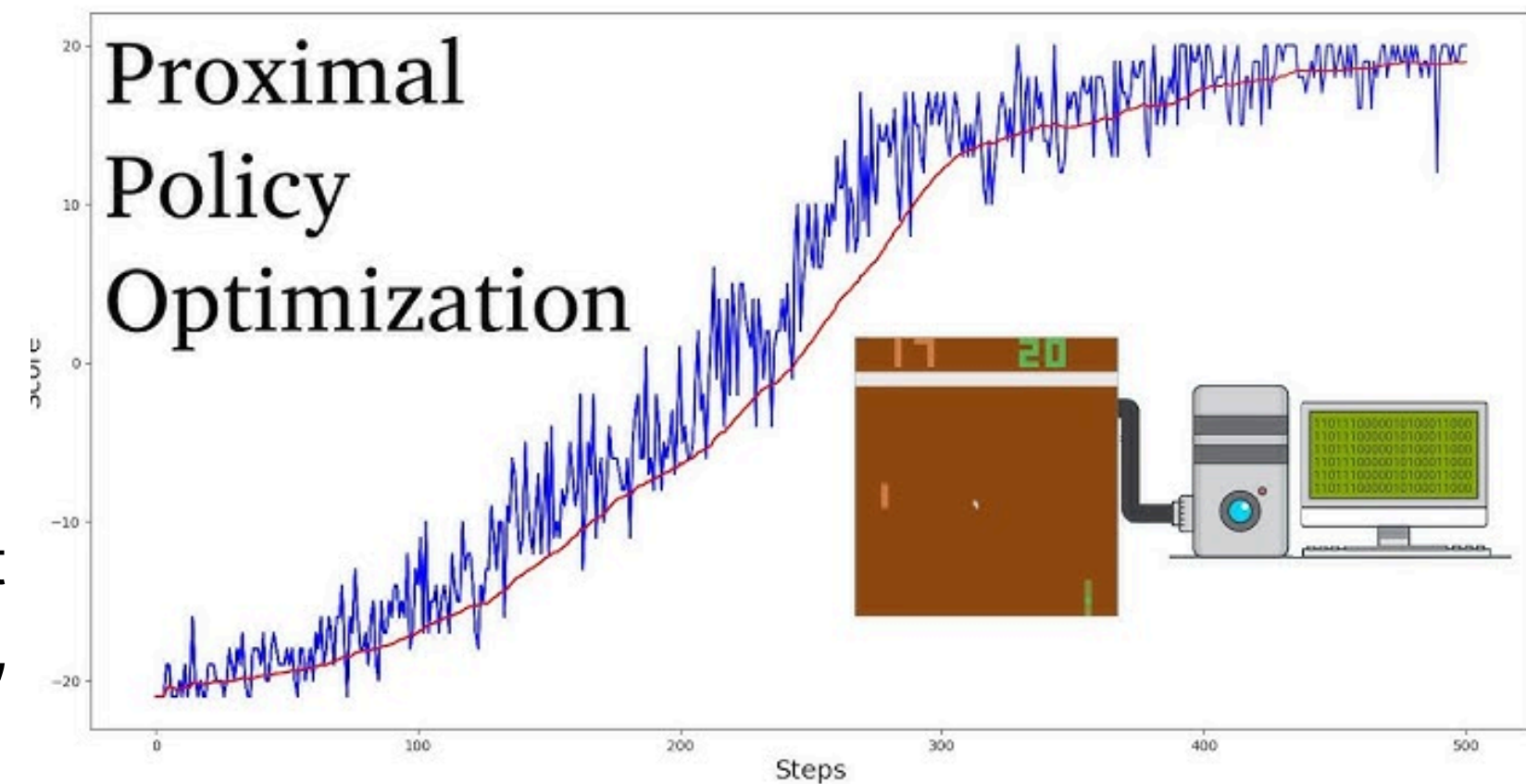
1. The training process leverages extensive parallel computing capabilities to simultaneously train multiple robots. This approach accelerates data collection and speeds up the learning process significantly.
2. The training environment gradually increases in difficulty based on the robot's performance, ensuring continuous challenge and skill development.
3. Robots navigate different terrain types—flat, sloped, rough, and obstacle-laden, providing a comprehensive training experience.



HOW PPO FITS INTO TRAINING THE A1 ROBO DOG

1. Policy Optimization with Clipping

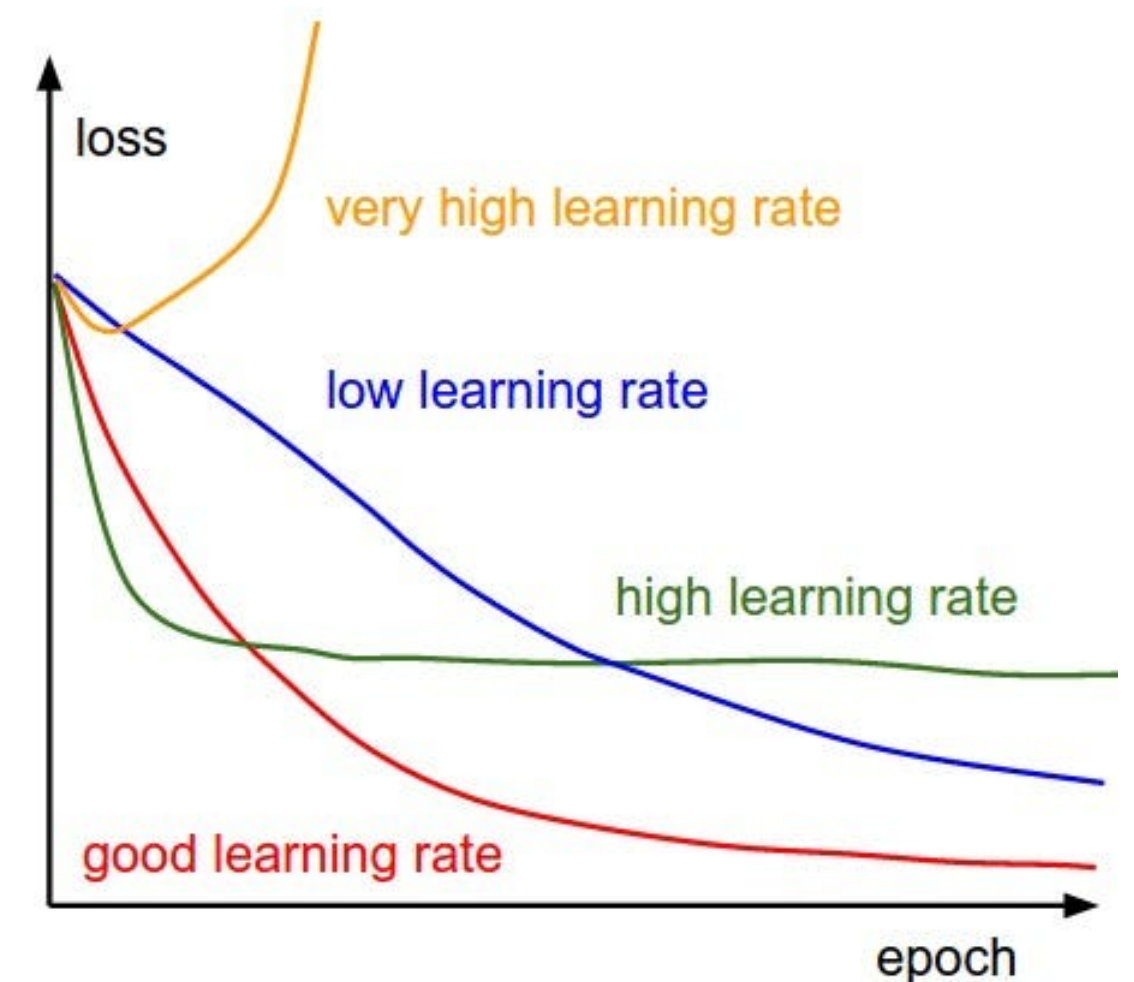
- PPO optimizes the robot's actions using clipping. This prevents large updates to the policy and ensures that actions don't change drastically between iterations.
- In the context of the A1 Robo Dog, this means that if the robot learns a slight improvement in walking or balance, it won't overcorrect or suddenly alter its movement pattern, reducing the risk of falls or inefficient movements.



HOW PPO FITS INTO TRAINING THE A1 ROBO DOG

2.Adaptive KL Divergence

- Kullback-Leibler (KL) divergence is a statistical measure that quantifies how one probability distribution differs from another. In reinforcement learning, it is often used to compare the old policy (the one currently being used) with the new policy (the one being updated).
- PPO changes its learning rate based on the stability of the policy, helping to maintain control over the training process.



HOW PPO FITS INTO TRAINING THE A1 ROBO DOG

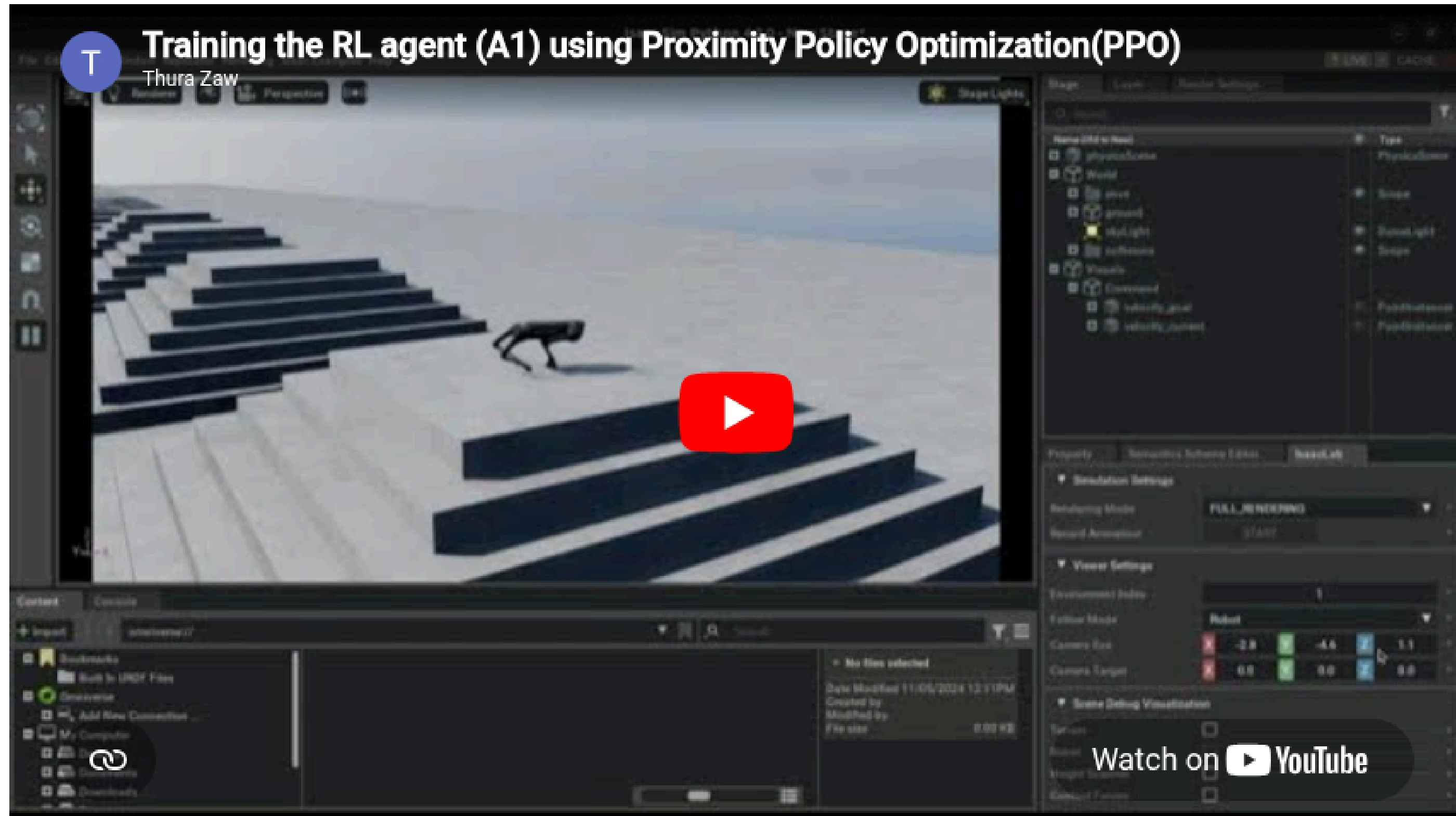
3. Generalized Advantages Estimation

- Generalized Advantage Estimation (GAE) is designed to compute more stable and less noisy advantage estimates for policy gradient algorithms. Proposed by John Schulman and colleagues, GAE helps balance bias and variance in advantage estimation, resulting in more sample-efficient and stable training.

What are in GAE

- Collect Experiences
- Calculate Returns
- Compute Advantages
- Control Bias-Variance Trade-off
- Update the Policy

WALKING RESULT



THANK YOU!